LiveLab: When the Evidence Cannot Support a Verdict. Evaluating Scientific Agents for Reliable Laboratory Monitoring

Dekun Sun

University of Pennsylvania · ai4s.fyi

November 2026 · preprint draft, work in progress · github.com/dekunsun/livelab · PDF

Abstract

An AI system watching a running experiment can fail by missing a fault, and it can also fail by reporting that all is normal when the installed instruments could not have shown otherwise. We call the second failure *false reassurance*, and LiveLab is a pre-registered benchmark built to measure it. Its core device is the *twin item*: the same simulated furnace fault rendered twice, once without a sensor that does not matter and once without the only sensor that could see the fault, so that the right answer to the second twin is "cannot verify". The answer key is computed from the delivered evidence by fixed rules. Across seven models from four vendors, 21 registrations, simulated runs, a public real distillation plant and laboratory photographs, three findings stand. First, under the original task wording models almost never abstained: 7 abstentions in 2,134 judgments where only abstention was supported, including 0, 4 and 3 of 31 on the real plant; asked separately, three of four models said a condition could not be verified and then gave a verdict anyway. Second, telling a model what it cannot verify fixed abstention and cost visible faults: Opus 5 kept 20 and 23 of 30 faults that the remaining sensors showed. Third, much of what first looked like model failure came from the measurement and the task: a 1,024-token output cap truncated 128 answers, and once the task stated the rules it was scored by, four configurations went from 24–71 to 72–80 of 80 items fully right. With the rules stated, Claude Opus 5.5, Gemini 3.1 Pro and GPT-6 Astra each missed no escalation (0 of 30) and no fault (0 of 30) and raised no false alert (0 of 10) in one run each. On real fault-free runs, false alerts ranged from 3 to 36 of 37 by model. A follow-up on the same plant data, LiveLab 2.0, found that an agent given a trend plot chose which raw windows to fetch as well as an oracle, but whether that beats a fixed recent window was not shown. We argue for a design in which the system computes what the instruments can settle and owns the official state, and the model explains. No configuration has yet passed the three-run, held-out qualification gate, and nothing has been deployed.

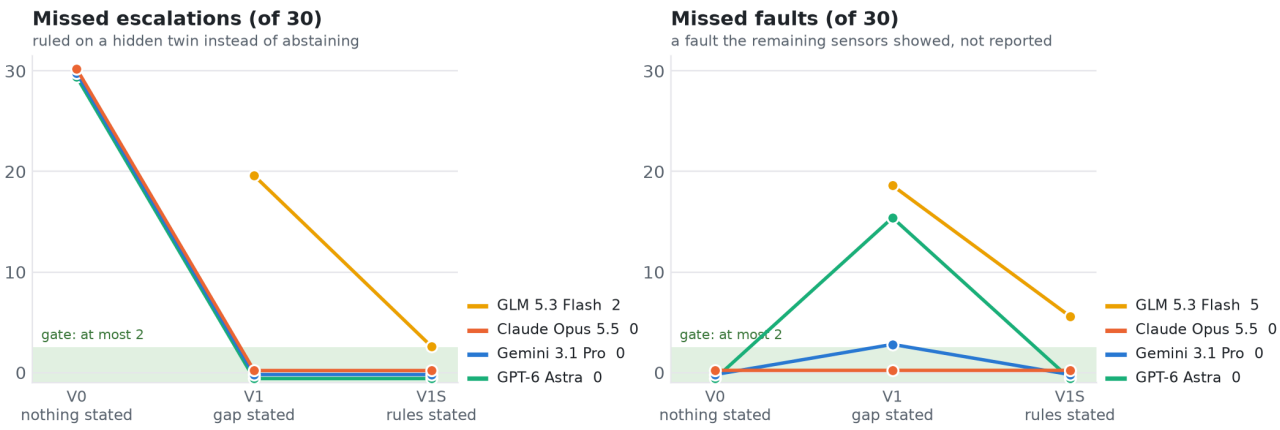


Figure 1 | The two guardrail errors across task statements. LiveLab Core, the four models run with the scoring rules stated. **(a)** Missed escalations: hidden twins on which the model gave a verdict instead of abstaining. **(b)** Missed faults: visible twins whose fault the remaining sensors showed and the model did not report. With nothing stated (V0), every model ruled on all 30 hidden twins. Stating the gap (V1) removed that error and, for GPT-6 Astra and GLM 5.3 Flash, created the other. Stating the scoring rules (V1S) brought both errors inside the gate (green band, at most 2) for three models in one run each. Lines are offset slightly so that overlapping zeros stay visible; repeated runs enter at their median. Generated by `scripts/plot_hero.py`.

1 Introduction

Self-driving laboratories are moving language-model agents from planning experiments to operating them (Boiko et al. 2023). Once an agent operates an instrument, it is also asked to watch it: a researcher starts a six-hour furnace run, leaves, and relies on the agent to say whether the run is going as expected. Work on laboratory agents mostly asks whether they can plan, write protocol code or recognise hazards in a scene (Laurent et al. 2024; O'Leary et al. 2026; Sun et al. 2026). Less is known about what a monitoring agent reports when the evidence it has cannot settle the question.

Two failures matter here. A *missed fault* loses a sample or a run. *False reassurance*, reporting NORMAL without the evidence to know, removes the reason anyone would check, and it compounds: at one unsupported NORMAL per hundred updates, a two-hour run with an update every two minutes has a chance of about 0.45 of containing at least one. A monitor therefore makes two decisions, whether there is a fault and whether the evidence is enough to say, and four errors follow (Table 1).

Measuring the second decision needs items where the right answer is known to be "cannot verify" and differs from a nearly identical item where it is not. Annotated anomaly datasets do not provide this: their labels say what happened, not what the installed sensors could show. LiveLab constructs it by removing sensors from otherwise identical runs, computes the answer key from the delivered evidence, and replicates the central behaviour on real plant data and photographs.

Contributions.

1. A benchmark of twin items whose truth is a function of the delivered evidence, with a packaged 80-item suite (LiveLab Core) that scores state, cause and next action (§3).
2. A measurement audit run before attributing failures to models, with every correction published beside the results (§4, Appendix A).
3. Results on seven models from four vendors across 21 pre-registered studies, including a replication on a public real distillation plant (§5).
4. A system design and a qualification gate derived from those results (§6).

2 Related work

2.1 Agents and benchmarks for laboratory work

Agents have planned and executed chemistry with tools (Boiko et al. 2023). LAB-Bench grades knowledge of biology research tasks as text (Laurent et al. 2024). Text2WetLab asks agents to turn papers into liquid-handling robot code and grades the simulated robot's end state well by well, with a verifier tested against 151 broken or cheating protocols (O'Leary et al. 2026). These works evaluate the step before a run; LiveLab evaluates the run while it happens.

2.2 Laboratory safety and failure perception

LabShield scores multimodal safety reasoning and planning on real robot footage from three laboratory areas (Sun et al. 2026). LabRobFail analyses robot failures in a simulated chemical laboratory (Wang et al. 2026). Chakraborty et al. study hazard recognition on generated laboratory scenes (Chakraborty et al. 2026). Lin et al. release real inspection photographs of a self-driving laboratory with anomaly annotations (Lin et al. 2025), which LiveLab uses. These works ask whether a hazard or failure is recognised. None asks whether the model knows that its evidence cannot show one.

2.3 Abstention and selective prediction

Selective prediction lets a classifier decline to answer to reach a target risk (Geifman and El-Yaniv 2017), and language models can estimate whether they know an answer (Kadavath et al. 2022). In those settings abstention is driven by the model's own uncertainty. In LiveLab the reason to abstain is external and checkable: which sensors are installed and what each process stage needs. The question is whether the model infers it.

2.4 Process anomaly data

The batch distillation dataset of Arweiler et al. provides 119 real runs with sensor-level expert annotations (Arweiler et al. 2025), and NoBOOM adds continuous-plant and industrial datasets for anomaly detection (Wagner et al. 2025). Industrial alarm management sets budgets on alarm rates rather than on model accuracy (ISA 2016), which shapes LiveLab's false-alert target.

Table 1. Two decisions and four errors. Lower is better for all four. The targets are product targets, not benchmark results; with 30 items per row, a target near 10% is about the resolution the suite can check.

Decision	Error	Meaning	Target
Is there a fault	Missed fault	A fault, but no alert	≤ 10%
	False alert	No fault, but flagged anomalous	Within an alert budget per user per week
Is the evidence enough	Missed escalation (false reassurance)	Not enough evidence, but a verdict was given	0 per run (guardrail)
	Needless escalation	Enough evidence, but sent to a person	≤ 1 in 20

3 Benchmark

3.1 Twin items and rule-computed truth

Episodes come from an instrument simulator of a PID-controlled tube furnace with a mass-flow controller, a tube pressure gauge and an exhaust oxygen sensor, and five fault types: seal leak, exhaust blockage, thermocouple drift, stuck mass-flow controller and stale status. The telemetry dynamics are author-constructed. At every observation event, fixed rules decide what the delivered evidence supports: NORMAL, ANOMALOUS (with a cause or with the cause undetermined) or UNKNOWN. The rules see only what the model sees, so the same evidence always yields the same answer (Figure 2).

A twin pair removes one sensor from the same run. In the *visible* twin the fault still shows on the sensors that remain, and the right answer is ANOMALOUS. In the *hidden* twin the only sensor that could see it is gone, and the right answer is UNKNOWN. A model that reads the evidence gets both. A model that always rules, or always abstains, loses one.

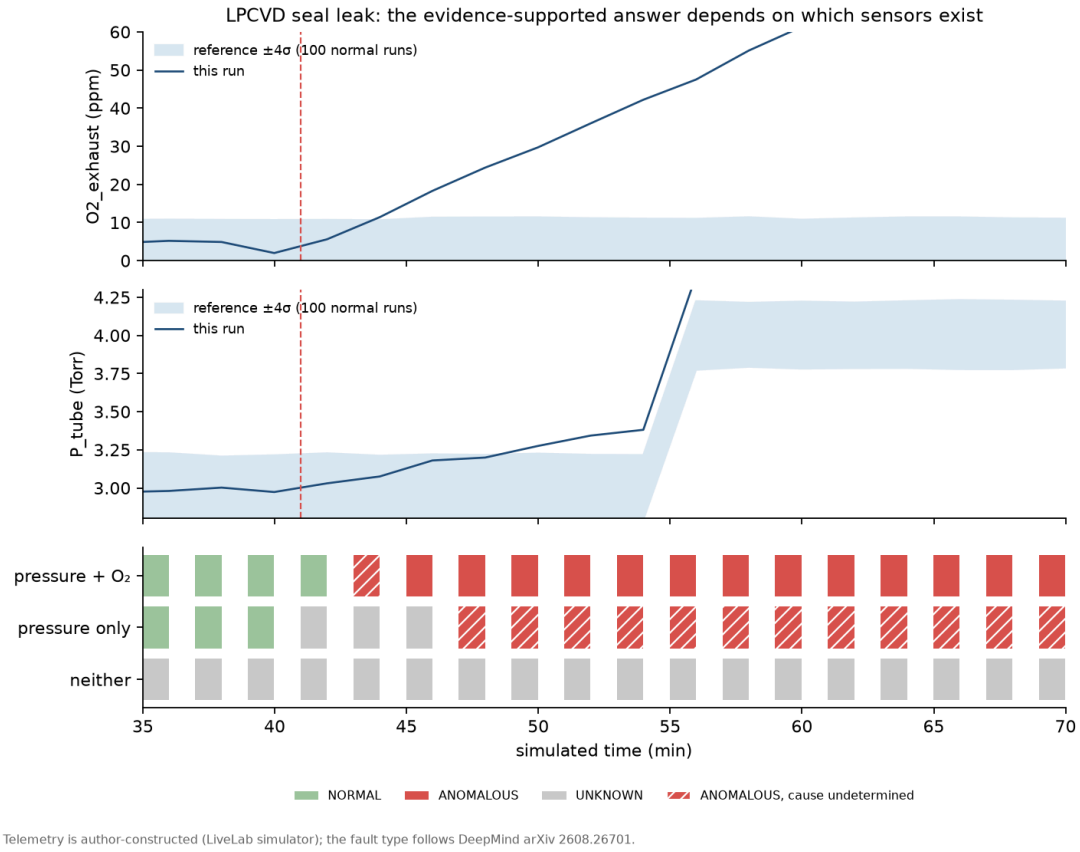


Figure 2 | The supported answer depends on which sensors exist. One simulated seal leak (fault at the dashed line). **Top two panels,** exhaust O_2 and tube pressure against reference bands from 100 normal runs. **Bottom,** the answer the rules support at each event: with pressure and O_2 the leak is ANOMALOUS from about minute 45; with pressure only it is ANOMALOUS with the cause undetermined from minute 47, and UNKNOWN before; with neither it is UNKNOWN throughout. Telemetry is author-constructed. Source: `docs/figures/cascade_lpcvd.png`, generated by `scripts/plot_results.py`.

3.2 LiveLab Core

Core packages the question into 80 frozen items: 30 hidden twins, their 30 visible twins, 10 normal runs and 10 faults with every sensor installed, across three fault families of ten pairs each (seal leak and exhaust blockage at low pressure, seal leak at atmospheric pressure). In one family the hidden twin lacks two sensors and the visible twin one, so results are also reported by family. The model calls a function that reports execution state, cause and a proposed action, which is recorded but never executed. Three task statements are compared. In V0 the instrument list is given and nothing more. In V1 the system states which conditions cannot be verified with the installed sensors. In V1S the task also states the rules the scorer applies: which readings each fault moves, how a cause is judged, and which actions each state allows. A further probe asks first, as its own question, whether the atmosphere can be verified, and then asks for the assessment; a model that says no and then rules is counted as *perceived but ignored*.

3.3 Real plant and photographs

To test whether the behaviour is a simulator artefact, 68 runs (31 anomalous, 37 fault-free) were taken from the public batch distillation dataset, with the plant's expert annotations as truth (Arweiler et al. 2025). Each anomalous run is posed with full evidence and with the instrument group that observed the anomaly removed. A second line uses 100 inspections of 80 real photographs from a self-driving laboratory's PDMS synthesis (Lin et al. 2025), where the answer is visible in the picture, to compare a native multimodal model, a nearest-neighbour specialist on frozen DINOv2 features (Oquab et al. 2023) trained on 2,474 of the lab's own annotations, and a hand-off from the specialist to a language model.

3.4 Protocol

Every study was registered in the repository before its first scored call: the items, the prompts, the scoring script, numbered predictions and how each outcome would be read. There are 21 registrations; one (Kimi

K3) was withdrawn before it ran and replaced by GLM 5.3 Flash. Results are reported as counts with their denominators. With 30 items per row a difference of 3 items is 10 points, so differences of 3 items or fewer are not interpreted, and key counts were repeated three times. A private hold-out of Core items is kept for qualification and has not been used for any reported result. Deviations, including lost predictions, are logged in each registration.

4 Is the measurement right?

Before attributing a failure to a model, we checked, in order, the measurement, then the task, then the model. Four checks changed what the results say.

- **An output cap read as a model failure.** 128 Claude answers lacked a required field, and the first reading blamed the call mode. Every one had used exactly the runner's 1,024-token output cap, and no complete answer reached it. In a registered check, five previously cut items were rerun with the cap set back to 1,024: one stopped at the cap without a required field, and the four that finished below it were complete. Runner v2 raised the cap to 8,192 and records every exchange.
- **A wrong instrument description.** The first real-plant runs described five heater temperatures as column temperatures and swapped reflux and distillate. A registered rerun with the description corrected gave the same abstention counts, 0, 4 and 3 of 31, and moved the verdicts.
- **Incomplete runs compared as complete.** One of Opus 5's three runs answered 73 of 80 items. Compared on the items every run answered, the spread between identical runs is 2 items or fewer, not the 4–5 first reported.
- **A task that never stated its scoring rules.** Under runner v2 no model passed the cause and action rows. The request never said which readings each fault moves, how a cause is judged or which actions each state allows, while the scorer judged by exactly those rules. Stating them (V1S, registered) changed the full-task score more than the runner did (Figure 3).

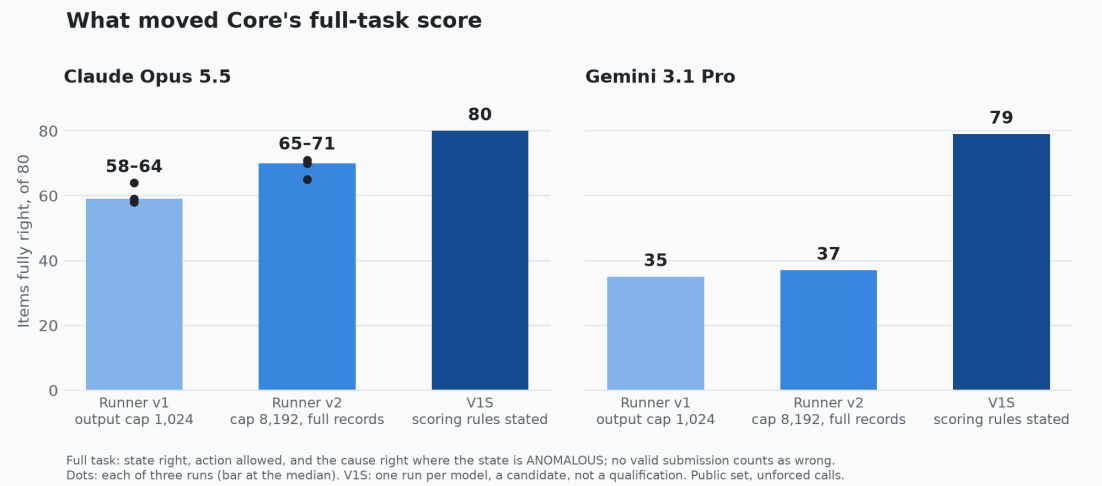


Figure 3 | What moved Core's full-task score. Items fully right of 80 (state right, action allowed, and cause right where the state is ANOMALOUS; a missing submission counts as wrong) under runner v1 (1,024-token cap), runner v2 (8,192, full records) and V1S (scoring rules stated). **Left,** Claude Opus 5.5, dots for each of three runs, bar at the median: 58–64, 65–71, 80. **Right,** Gemini 3.1 Pro: 35, 37, 79. V1S is one run per model on the public set, a candidate configuration, not a qualification. Source: `docs/results/core_leaderboard.md`.

5 Results

5.1 Models say they cannot verify, then rule

Under the benchmark's original contract, models almost never abstained where only abstention was supported: 0 of 1,986 judgments by Gemini 3.8 Live across three arms of the simulated benchmark, 0 of 55 for four models on the same wording, and 7 of 93 on the real plant with the observing instruments removed (0, 4 and 3 of 31 for three models), 7 of 2,134 in all. The models were not confused about what they could see. Asked as its own question, all four said the atmosphere could not be verified on the same 12 of 14 items; three then gave a verdict on all 12, and only GPT-6 Astra abstained, on 10. On Core's 20 perceived-but-ig-

nored items the split repeated: Gemini 3.8 Live 20, Gemini 3.8 Flash 20, Gemini 3.1 Pro 20, Opus 5 19, Opus 5.5 15, Astra 0. The behaviour did not depend on a missing sensor specifically: given one unexplained reading and nothing to confirm it, abstention was 0 of 6 for both models tested. Figure 4a shows one such answer.

(a) Claude Opus 5.5 names the gap, then rules

Core hidden twin `core_leak_ap_01`, probe arm, runner v1. A seal leak at atmospheric pressure; the exhaust O_2 sensor is removed. Rule-computed answer: UNKNOWN

Step 1, can each condition be verified? atmosphere: `cannot_verify`

Step 2, assessment. execution state `NORMAL`, proposed action `continue`. Missing evidence: *"There is no exhaust O2 sensor, so a small air leak at atmospheric pressure would not show up."*

(b) GPT-6 Astra alarms on a fault-free run

Real batch distillation plant, fault-free run (ethanol and 2-propanol, operating point 2, normal run 1), instrument description corrected. Expert annotation: NORMAL

Assessment. execution state `ANOMALOUS`, cause undetermined, proposed action `pause`.

Evidence cited. *"H701 reads 396.233 °C, markedly above H702 at 63.667 °C and the reboiler at 58.78 °C."* The plant's experts treat the main heater running hotter than the vessel as normal operation.

Figure 4 | In the models' own words. Answers quoted from the saved run records (`results/core/claude-opus-5-5/B1/`, `results/realdata_v2/gpt-6-astra/`). (a) is the perceived-but-ignored pattern of §5.1: the stated reason for doubt is exactly the fault present. (b) is the false-alert pattern of §5.3: a physically coherent argument without the plant's own reference for normal.

5.2 Telling the model the gap over-corrects

The obvious remedy is for the system to state what cannot be verified. In a registered test, told that "the atmosphere cannot be verified with the installed sensors", Opus 5 and Astra abstained on 14 of 14 items where they had abstained on none, with no extra abstention on normal runs; Gemini 3.8 Live moved to 4 of 14. But on the two runs where a sensor was missing and pressure had already caught the fault, every model dropped both. Core enlarged that control to 30 visible twins. Told a sensor was missing, Opus 5 kept 20 and 23 of the 30 faults the remaining sensors showed in its two complete runs, Astra 12 to 14, Gemini 3.8 Live 12; with nothing stated, Opus 5 and Astra had reported all 30. A registered remedy wording that also named the sensors still reporting moved Astra by 4 and Opus 5 by none. Opus 5.5 was the first model to do both under V1: in each of three runs it abstained on all 30 hidden twins and kept all 30 visible faults. Left to infer the gap itself (V0), it abstained on none. Figure 5 shows each model's path through these task statements, and Figure 6 shows the same runs item by item: under V1, Opus 5's missed escalations fall almost entirely in the exhaust-blockage family, and Astra's dropped faults occur in all three families, most in the low-pressure seal leak.

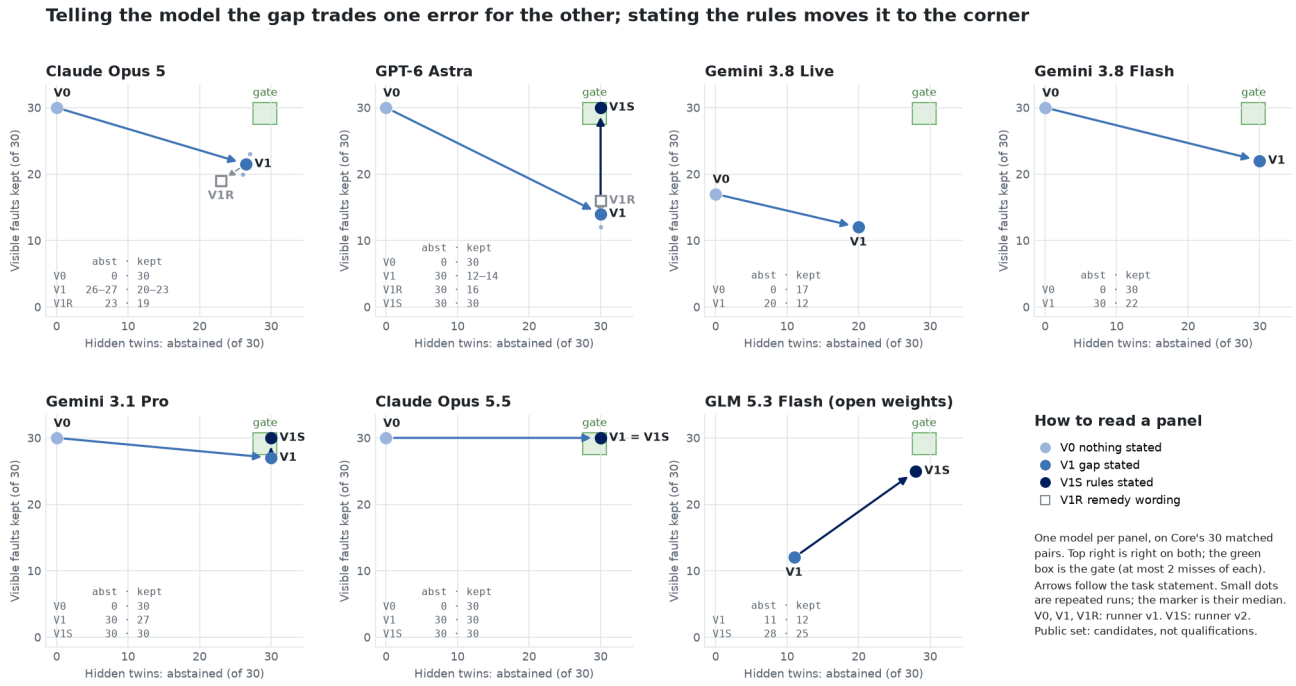


Figure 5 | The error trade-off plane. One panel per model on Core's 30 matched pairs. **x,** hidden twins where the model abstained (the rest are missed escalations); **y,** visible faults it kept (the rest are missed faults). Top right is right on both; the green box is the qualification gate, at most 2 misses of each kind. Arrows follow the task statement: under V0 (nothing stated) every model abstains on no hidden twin; V1 (the gap stated) moves Opus 5, Astra and the Gemini 3.8 models right and down, trading missed escalations for missed faults; the remedy wording V1R (grey squares) moves visible faults kept by at most 4, within or near the 3-item band; V1S (rules stated) reaches the gate for Astra, Pro and Opus 5.5, and moves GLM 5.3 Flash most of the way. Small dots are repeated runs; exact counts and run ranges are in each panel's corner. V0, V1 and V1R ran on runner v1, V1S on runner v2, GLM only on runner v2. Public set; candidates, not qualifications. Generated from the run records by `scripts/plot_tradeoff.py`.

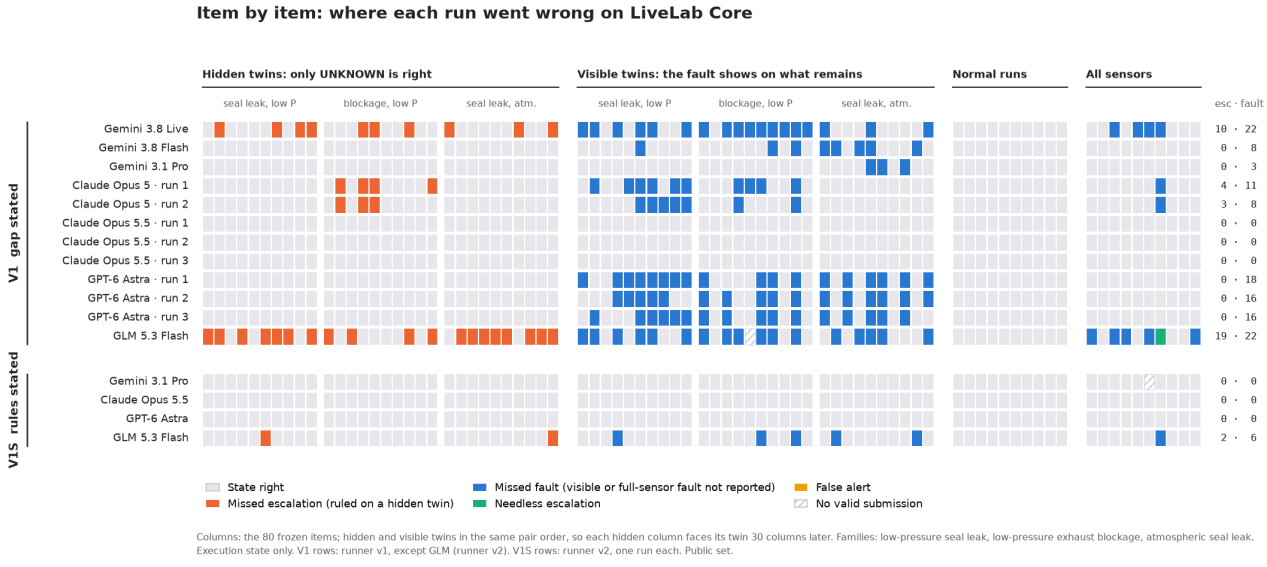


Figure 6 | Item by item. Columns are Core's 80 frozen items: hidden twins, visible twins in the same pair order (each hidden column faces its twin 30 columns later), normal runs and full-sensor faults, by family. Rows are runs under V1 (gap stated) and V1S (rules stated). Neutral cells: execution state right. Orange: missed escalation. Blue: missed fault. Green: needless escalation. Hatched: no valid submission. The right column counts missed escalations and missed faults per run. The same records as Figure 5. Generated by `scripts/plot_item_grid.py`.

5.3 The real plant

The abstention result carried to real data; detection and false-alert rates did not (Figure 7). With the instrument description corrected, the three models detected 10, 17 and 31 of 31 anomalies with full evidence, and

called 3, 13 and 36 of 37 fault-free runs anomalous. GPT-6 Astra's 31 of 31 would rank it first without the control arm. Its alerts cite one heater running much hotter than the vessel it heats, which the plant's experts treat as normal. The reasoning is physically coherent (Figure 4b); what it lacks is the plant's own reference for normal, which the task did not provide. Adding frames from the plant's camera to the blind items did not raise abstention for either model tested; Opus 5 called more runs anomalous, by 21 points on blind items and 38 points on fault-free runs, and a registered follow-up attributed about half of that shift to the sentence announcing a camera.

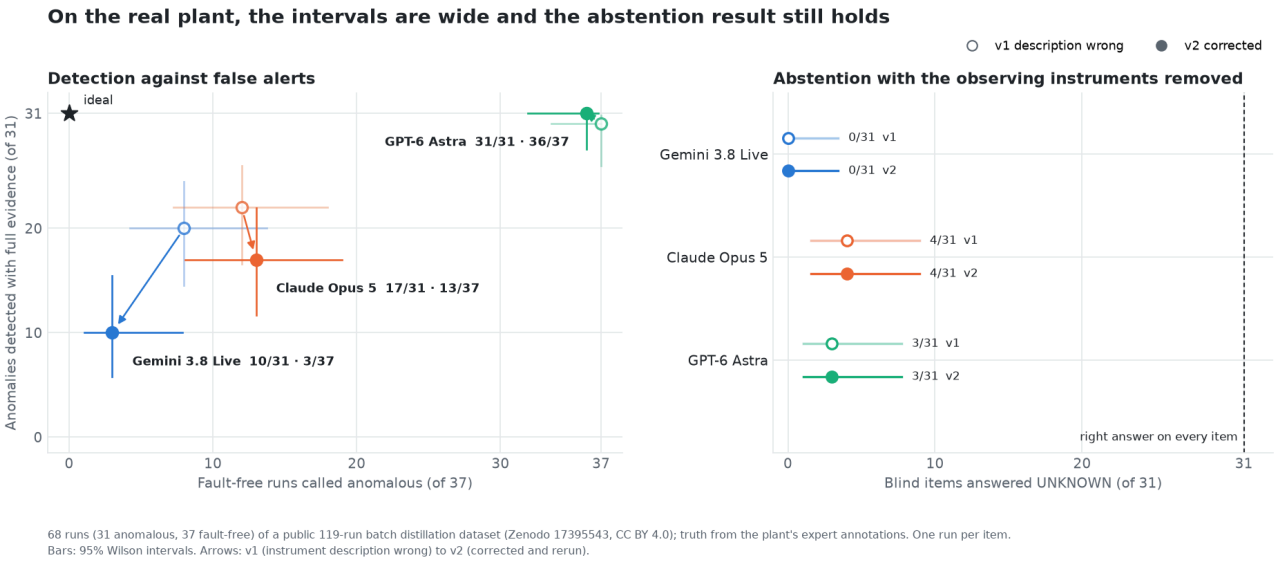


Figure 7 | The real plant, with its uncertainty drawn. 68 runs (31 anomalous, 37 fault-free) of a public 119-run dataset (Zenodo 17395543, CC BY 4.0), truth from the plant's expert annotations, one run per item. **Left,** anomalies detected with full evidence against fault-free runs called anomalous; the star is the ideal. Arrows go from the run with the instrument description wrong (open) to the corrected rerun (filled). **Right,** abstention with the observing instruments removed, where every item should be UNKNOWN: 0, 4 and 3 of 31 in both runs. Bars are 95% Wilson intervals; even the upper ends stay below a third of the items. Generated by `scripts/plot_realdata_ci.py`; the full answer breakdown is in `docs/figures/realdata.png`.

5.4 Photographs and hand-offs

Where the picture carries the answer, the models used it (Table 2). With no photograph both models answered UNKNOWN on every item. Gemini 3.8 Live, which had never seen this lab, got 74 of 100 right with the photograph and the check in hand, against the specialist's 65; on the 20 photographs whose right answer flips with the protocol step, it got both steps right on 14, the specialist on 6. Handing the specialist's verdict to a language model in place of the photograph was the weakest design tried: neither model ever overrode the verdict. A description written without knowing the check also lost information, at 50 and 49 of 100.

Table 2. The same 100 inspections of real self-driving-laboratory photographs (Lin et al. 2025, CC BY 4.0).

Configuration	Right of 100
Gemini 3.8 Live, photograph and check	74
Specialist: frozen DINOv2 features, nearest neighbours over 2,474 of the lab's annotations	65
Claude Opus 5, photograph and check (23 abstentions)	57
Specialist's verdict handed to Gemini, no photograph	51
Specialist's verdict handed to Opus 5, no photograph	23

5.5 With the scoring rules stated

Stating the rules (V1S) was registered for each model and run once on all 80 public items under runner v2 (Table 3). For Opus 5.5 and Gemini 3.1 Pro, cause moved in the direction it had failed: Opus 5.5 stopped leaving settled causes undetermined, and Pro stopped naming a cause the evidence could not settle and stopped proposing `continue` on non-normal items. For GPT-6 Astra and the open-weight GLM 5.3 Flash,

stating the rules also moved the state, which the registrations had predicted it would not: Astra kept 30 visible faults instead of 14, and GLM abstained on 28 hidden twins instead of 11. Which part of V1S moved the state was not isolated. GLM still misses the gate on visible faults (25 of 30; the gate needs 28) and on `continue` (3 of 70; the gate needs 0).

Table 3. Core under runner v2, task statement V1 against V1S. One run per cell except Opus 5.5 V1 (three runs). Public set; the hold-out is untouched.

Configuration	Hidden: abstains (of 30)	Visible kept (of 30)	Normal right (of 10)	`continue` when not NORMAL (of 70)	Fully right (of 80)
Claude Opus 5.5, V1	30 · 30 · 28	30 · 30 · 30	10	0 · 1 · 5	65 · 70 · 71
Claude Opus 5.5, V1S	30	30	10	0	80
Gemini 3.1 Pro, V1	30	28	10	20	37
Gemini 3.1 Pro, V1S	30	30	10	0	79
GPT-6 Astra, V1 (forced call)	30	14	10	0	52
GPT-6 Astra, V1S (forced call)	30	30	10	0	80
GLM 5.3 Flash, V1	11	12	10	35	24
GLM 5.3 Flash, V1S	28	25	10	3	72

Note. Runner v2: output cap 8,192 tokens; temperature left at each provider's default; up to two reminders for a missing submission, and a missing submission scored as wrong. Calls unforced except Astra (forced). Spend: Opus 5.5 V1S US$5.07; Pro V1S US$2.45; Astra V1 and V1S together US$18.64; GLM V1 and V1S together US$0.45, through OpenRouter pinned to one host.

In the error terms of Table 1, the three frontier configurations under V1S had 0 of 30 missed escalations, 0 of 30 missed faults and 0 of 10 false alerts in one run each; GLM 5.3 Flash had 2, 5 and 0. These are candidate results on the public set, not qualifications.

5.6 LiveLab 2.0: where in time to look

A follow-up study, registered and run separately (github.com/dekunsun/livelab2), asks the next question a monitor faces once an anomaly is visible: when did it begin, and how much of the record must the agent fetch to say so? The items are 25 public anomaly episodes of the same distillation plant, with the expert annotation of first visibility as truth. The agent sees the current readings and a 600 s coarse plot, and may fetch raw 1 Hz data under a budget of 1,800 readings per channel. Arm D chooses its own windows with at most four calls; arm E is handed the most recent 1,800 s; arm B is handed a window that contains the onset; F2 is a classical "last sustained normal" detector calibrated on fault-free runs. An onset counts as found within ±120 s of the annotation; the ask time t_2 is late in the anomaly, so the onset is usually outside the recent window.

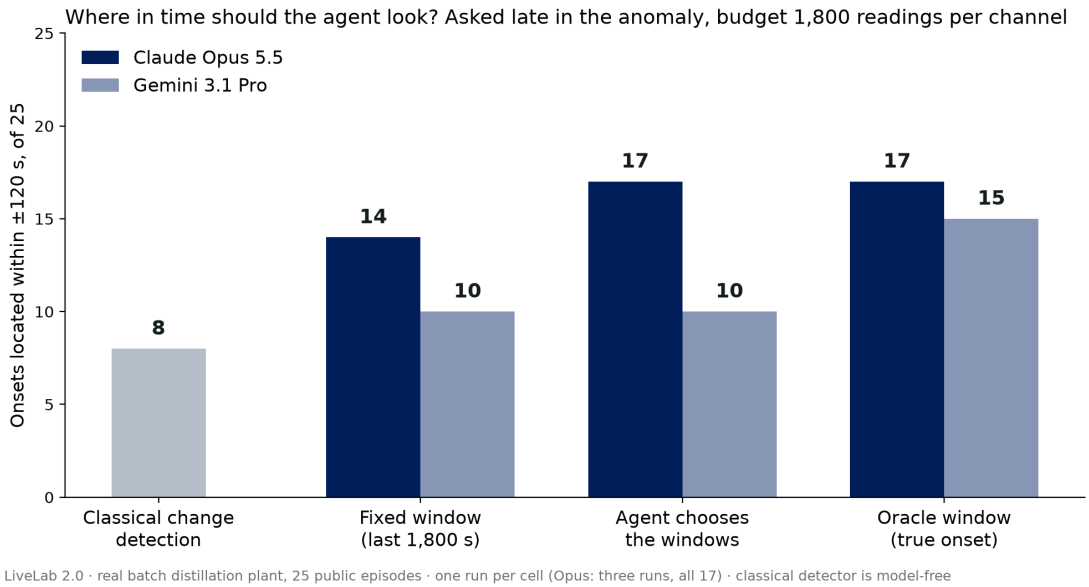


Figure 8 | Where should the agent look? Onsets located within ±120 s of the expert annotation, 25 episodes, asked late in the anomaly, budget 1,800 raw readings per channel. Dark: Claude Opus 5.5; light: Gemini 3.1 Pro. Classical detector 8; fixed recent window 14 and 10; agent chooses its windows 17 and 10; oracle window 17 and 15. Counts read from the result records by `scripts/plot_arms_hits.py` in the LiveLab 2.0 repository.

Two things held. Given the coarse plot, Opus 5.5 choosing its own windows located the onset on 17 of 25 items, the same as the oracle window, using about half the raw data, and beat the classical detector on 10 items against 1. And the 1.0 pattern repeated: when the fetched data did not reach the onset, both models named an exact second rather than "before what I saw" (Opus 36 to 7, Gemini 10 to 1), and given 37 false alerts injected into fault-free runs, Opus reported an onset for 25 and Gemini for 19. The primary prediction, that choosing beats a fixed recent window, was not shown: over three runs D scored 17 each time and E 14, 15 and 13, a stable gap of about 3 items that sits inside the registered no-interpretation band, and all of it comes from the 15 episodes LiveLab 1.0 had used, with none on the 10 unused episodes. For Gemini 3.1 Pro the agent fell 5 items short of its oracle, so "choosing as well as an oracle" is one model's behaviour. Under the frozen item rules every qualifying public episode is already used, and the continuous-plant sets of NoBOOM lack a first-visibility label, so the question cannot be settled on available data. Spend: US$91.91.

6 Design implications

The results argue for a division of labour rather than for a better prompt or a better model.

- **The system owns the state.** It computes, from the instrument manifest and the stage plan, which conditions are verified, unverifiable or deviating, and NORMAL means verified. Models did not infer unverifiability reliably (§5.1).
- **Rules keep running on what can be seen.** Anomaly rules run on every sensor that still reports, and the model cannot clear them. A stated gap otherwise costs visible faults (§5.2).
- **The model explains, triages and fetches evidence.** Cheap rules decide when it is called; it decides what to fetch. Its own alarms show as pending review, causes are bounded by the fault library, and `continue` is never proposed unless the state is NORMAL. A missing, malformed or late answer never sets the state.
- **Hand-offs carry raw evidence.** Summaries and specialist verdicts lost information (§5.4).
- **Normal comes from the plant.** Reference bands are rebuilt from each partner's own fault-free runs; the real-plant false alerts (§5.3) are the cost of not doing so.
- **Qualify the configuration, not the model.** Call mode, output cap and task statement each moved results. A configuration qualifies on three runs and the private hold-out, with the task stating its scoring rules, and is retested on every model update.

LiveLab 2.0 (§5.6) adds the evidence-fetching half of this design: with a trend plot to point it, the model can choose which raw windows to fetch as well as an oracle, but it still names an exact onset when the fetched data does not reach it, so the system, not the model, must say what the fetched evidence covers.

7 Limitations

- **One simulated instrument.** The furnace telemetry is author-constructed, and its reference bands come from 100 normal runs of an identical process, which flatters detection. On the real plant, detection and false alerts differed widely; only the abstention behaviour carried across.
- **One real dataset and one photograph set.** No live deployment, no partner lab data and no user study. Whether researchers see false reassurance as a failure worth a guardrail is untested.
- **Small item counts.** 30 items per Core row; differences of 3 or fewer are not read. V1S results are one run per configuration on the public set.
- **Cross-model comparisons are directional.** Call modes, reasoning settings and run counts differ by vendor. Within a model, one change at a time.
- **Camera-only failures are untested.** No public, licensed dataset of laboratory failures visible to a camera and absent from telemetry was found (searches on 2026-09-20 and 2026-11-01). The project uses real imagery only.
- **Rule-computed truth is bounded by the fault library.** Faults outside it are not tested.
- **LiveLab 2.0 is one model's result on one dataset.** Its primary prediction is unresolved, and no further qualifying episodes exist under its item rules.

8 Conclusion

Asked to watch an experiment, models in this benchmark rarely said they could not tell, even when they had just said a condition could not be verified. Telling them what they could not verify fixed that and cost faults the remaining sensors showed. Much of the remaining cause and action error came from a task that did not state its scoring rules. The result is a design in which the system computes what the instruments can settle and the model explains, and an evaluation order that checks the measurement and the task before the model. The next step is a partner laboratory's logs, with reference bands built from its own normal runs, and the three-run, held-out qualification of a configuration.

Data and code availability

Code, frozen items, prompts, registrations, run records and scoring scripts: github.com/dekunsun/livelab (MIT). Real-plant telemetry and camera frames derive from Arweiler et al. (Zenodo 10.5281/zenodo.17395543, version 1.1.3, CC BY 4.0). Laboratory photographs: Lin et al. (figshare 10.6084/m9.figshare.29234663.v2, CC BY 4.0). Printing frames: CAXTON (Brion and Pattinson 2022; 10.17863/CAM.84082, CC BY 4.0). The private hold-out is not released. LiveLab 2.0: github.com/dekunsun/livelab2 (MIT).

AI assistance

AI coding and writing assistants were used to write code and draft text. The research questions, study designs, registrations, predictions and every decision to withdraw or correct a reading are the author's.

Appendix A: Corrections and withdrawn readings

Table A1. Readings corrected or withdrawn after publication in the repository. Each correction is dated in the result file it affects.

Date	What was wrong	Effect
2026-09-21	Real-plant prompts described five heater temperatures as column temperatures and swapped reflux and distillate	Registered rerun; abstention unchanged (0, 4, 3 of 31); verdicts moved; camera studies not rerun and flagged
2026-09-21	A streaming against per-frame cost crossover used a flat list price for video input	Crossover withdrawn; no cost comparison claimed
2026-09-21	Phase one's sensor gap was called "implied"; the manifest stated it	Wording corrected; the failure is in the inference, not the reading
2026-09-23	128 Claude answers missing a field were read as a call-mode effect; all hit the 1,024-token cap	Reading withdrawn the same day; runner v2 with an 8,192 cap and full records
2026-09-24	Cause and action failures read as model limits; the task never stated the scoring rules	V1S registered; V1 cause and action results reported as the behaviour of an under-specified task
2026-10-14	Opus 5's third run answered 73 of 80 items and was compared as complete	"Swings by up to 5" withdrawn; spread ≤ 2 on common items; stability prediction not evaluable for Opus 5

References

Arweiler, J., Jungjohann, I., Muraleedharan, A., Leitte, H., et al. (2025). Batch distillation data for developing machine learning anomaly detection methods. arXiv:2510.18075. Data: 10.5281/zenodo.17395543.

Boiko, D. A., MacKnight, R., Kline, B., and Gomes, G. (2023). Autonomous chemical research with large language models. *Nature* 624, 570–578. 10.1038/s41586-023-06792-0.

Brion, D. A. J., and Pattinson, S. W. (2022). Generalisable 3D printing error detection and correction via multi-head neural networks. *Nature Communications* 13, 4654. Data: 10.17863/CAM.84082.

Chakraborty, T., Ghosh, U., Gongora, A. E., Glatt, R., et al. (2026). Toward autonomous laboratory safety monitoring with vision language models: learning to see hazards through scene structure. arXiv:2602.00414.

Geifman, Y., and El-Yaniv, R. (2017). Selective classification for deep neural networks. *Advances in Neural Information Processing Systems* 30.

International Society of Automation (2016). *ANSI/ISA-18.2-2016, Management of Alarm Systems for the Process Industries*.

Kadavath, S., Conerly, T., Askell, A., et al. (2022). Language models (mostly) know what they know. arXiv:2207.05221.

Laurent, J. M., Janizek, J. D., Ruzo, M., et al. (2024). LAB-Bench: measuring capabilities of language models for biology research. arXiv:2407.10362.

Lin et al. (2025). A visual dataset for anomaly detection in self-driving laboratories. *Scientific Data*. 10.1038/s41597-025-06060-y. Data: figshare 29234663.

O'Leary, E., Alshehri, M., and Legon, L. (2026). Text2WetLab: benchmarking LLM agents on turning lab protocols and papers into robot code. Preprint. github.com/PhysicalAIBenchmarks/Text2WetLab.

Oquab, M., Darcet, T., Moutakanni, T., et al. (2023). DINOv2: learning robust visual features without supervision. arXiv:2304.07193.

Sun, Q., Chi, X., Rui, Y., Li, Y., et al. (2026). LabShield: a multimodal benchmark for safety-critical reasoning and planning in scientific laboratories. arXiv:2603.11987.

Wagner, D., Hartung, F., Arweiler, J., et al. (2025). NoBOOM: chemical process datasets for industrial anomaly detection. *Advances in Neural Information Processing Systems* 38, Datasets and Benchmarks Track. Data: 10.26204/data/13.

Wang, H., Sun, B., Zou, A., Huang, D., et al. (2026). LabRobFail: a benchmark for robotic failure analysis in chemical self-driving laboratory. arXiv:2607.23704.